```
Output:
The ace of spades, the ace of spades \m/
```

This isn't a terrible approach; we can test and assign data against the constants `Suit::CLUBS`, `Suit::HEARTS`, etc. and make our code safer, but only up to a point. The type checking done in `Card::__construct()` is on the values of the `Suit` constants, not the class itself. If we add the following method to `Card`…

```
    public function getSuit(): string
    {
        return $this->suit;
    }
```

… and do something silly like this …

```
$card = new Card("cabbages", "four");

switch ($card->getSuit()) {
    case Suit::CLUBS:
    case Suit::DIAMONDS:
    case Suit::HEARTS:
    case Suit::SPADES:
        // carry on with the game
        break;
    default:
        throw new \Exception('Preposterous!');
}

Output:
Fatal error: Uncaught Exception: Preposterous!
```

… we break our code. We could add some more code to the `Card` constructor to check the incoming value is one of the four constants of `Suit`, but that's old PHP. Here's how to solve the problem with an enum.